VulnHub Machine: Primer: 1.0.1

Attacker Machine: Kali Linux
(192.168.56.104)
Target Machine: PRIMER (192.168.56.101)
From
Saniya Shaikh

Contents

1. Objective 3
2. Lab Environment 3
3. Target Information 3
4. Reconnaissance 4
   Step 1 - Identify the attacker interface and target on the lab network 4
   Step 2 - Perform a full service scan 4
   Step 3 - Open the PRIMER landing page 5
   Step 4 - Enumerate the web server with Nikto 6
5. Hidden Web Paths and Chapter Progression 7
   Step 5 - Read robots.txt 7
   Step 6 - Open the disclosed chapter 7
   Step 7 - Enter the __init__ chapter 7
   Step 8 - Enter the FORK chapter and identify Node 6 8
6. Client-Side Password Gates 9
   Step 9 - Bypass the Node 6 JavaScript password check 9
   Step 10 - Observe the Node 6 result page 9
   Step 11 - Inspect Node 7 source for the second password 9
   Step 12 - Enter the Node 7 password 10
   Step 13 - Confirm the Node 7 success page 11
7. Recognizing the Prime-Number URL Pattern 12
   Step 14 - Recover the hidden numeric pattern from the MD5 suffixes 12
   Step 15 - Open the next node and reach the simulated terminal 12
8. Pivot to Erebus 14
   Step 16 - Use process information to discover the next connection 14
   Step 17 - Authenticate to falken@Erebus 14
   Step 18 - Enumerate Erebus and inspect falken's files 14
   Step 19 - Review and decode the falken log data 15
9. Pivot to TrivialZ3r0 17
   Step 20 - Authenticate to falken@TrivialZ3r0 17
   Step 21 - Enumerate the password store on TrivialZ3r0 17
   Step 22 - Recover the chaos password 18
10. Pivot to Wintermute and Recover the Final Credentials 19
   Step 23 - Connect to Wintermute 19
   Step 24 - Enumerate the final Wintermute environment 19
   Step 25 - Read the final credentials from nieve 20
   Step 26 - Complete the PRIMER sequence 20
11. Credential Summary 22
   Why these passwords work 22
12. Conclusion 22

1. Objective

The objective of this lab was to work through the VulnHub PRIMER 1.0.1 challenge in sequence, starting with basic network and web enumeration and then following the machine's story-driven puzzle chain until the final credentials were exposed. The challenge mixes ordinary penetration-testing tasks with application logic, client-side JavaScript checks, simple hash analysis, and a simulated terminal.

This report follows the screenshots in the supplied Primer.docx in the same order they appear. Each step explains what is visible, what the action means in simple language, what the result tells us, and why that result leads directly to the next stage. Where a screenshot does not show a literal command, the explanation describes the visible clue rather than inventing an unseen action.

2. Lab Environment

- Attacker machine: Kali Linux (user: saniya).
- Attacker IP address: 192.168.56.104.
- Target machine: VulnHub PRIMER 1.0.1.
- Target IP address: 192.168.56.101.
- Network: isolated VirtualBox lab network on 192.168.56.0/24.
- Primary tools visible in the evidence: nmap, Nikto, browser/developer tools, curl, MD5 analysis, and the challenge's simulated terminal.

3. Target Information

Item	Value
Attacker IP	192.168.56.104
Target IP	192.168.56.101
22/tcp	OpenSSH 6.7p1 Debian 5
80/tcp	Apache httpd 2.4.10 (Debian)
111/tcp	rpcbind
37120/tcp	status

4. Reconnaissance

Reconnaissance is the first stage because it prevents us from guessing. We first establish which machine is ours, which machine is the target, and which services are reachable.

Step 1 - Identify the attacker interface and target on the lab network

Command : ip addr and sudo nmap -sn 192.168.56.0/24

Description: The first screenshot shows the Kali interface information and a ping-style Nmap sweep of the 192.168.56.0/24 network. The active interface eth1 owns 192.168.56.104/24, and the discovery scan reports the PRIMER host at **192.168.56.101**.



Figure 1: Kali IP configuration and initial target discovery

Result: The attacker is 192.168.56.104 and the challenge VM is 192.168.56.101.

Interpretation: The two systems are on the same private lab network, giving the attacker a direct path to the target.

Why this matters: Every later web request, scan, and browser connection needs the correct target address.

Next step: Run a service scan against 192.168.56.101.

Step 2 - Perform a full service scan

Command / Action: sudo nmap -sC -sV -p- --min-rate 5000 192.168.56.101

Description: The Nmap screenshot shows a full TCP service/version scan. The goal is to identify reachable services and their versions.

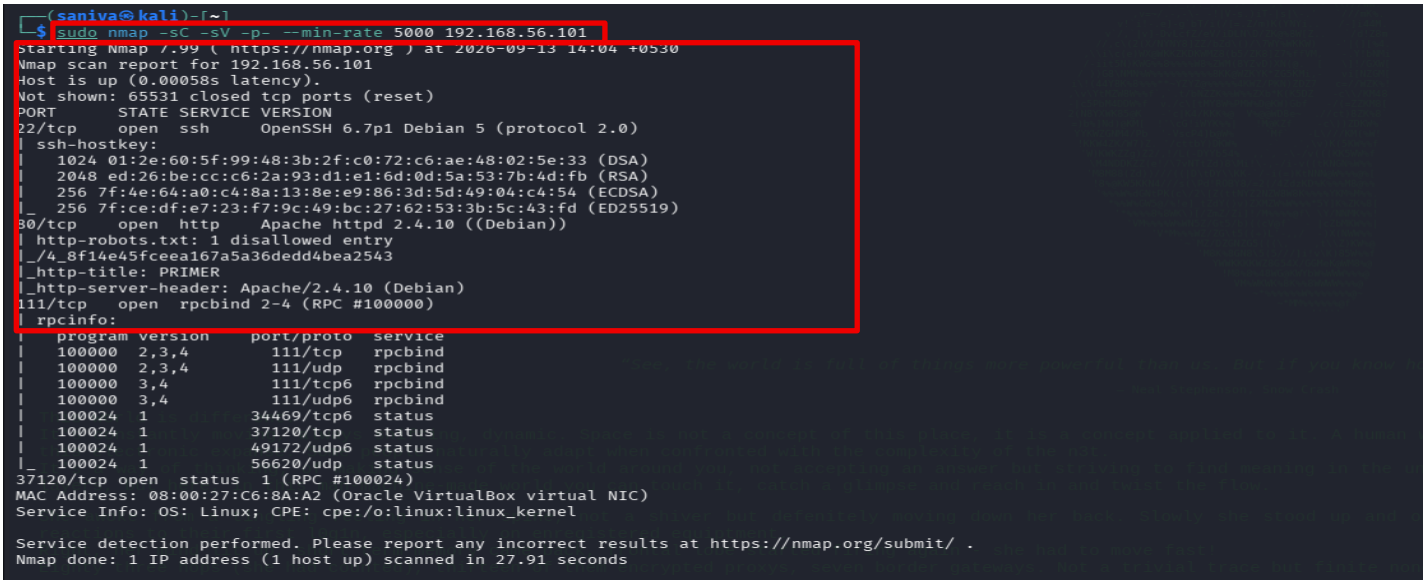


Figure 2: Nmap service and version scan of PRIMER

Result: Open TCP ports are 22, 80, 111, and 37120. Port 80 is Apache 2.4.10 on Debian and the title is PRIMER.

Interpretation: HTTP is the clearest attack surface because the browser can interact with it directly and the challenge is delivered through the website.

Why this matters: A concrete service list prevents random probing.

Next step: Open the web application and inspect it before automated enumeration.

Step 3 - Open the PRIMER landing page

Command: Browse to http://192.168.56.101

Description: The homepage is a custom PRIMER interface with green text, a story, and no conventional credential pair visible on the screen.

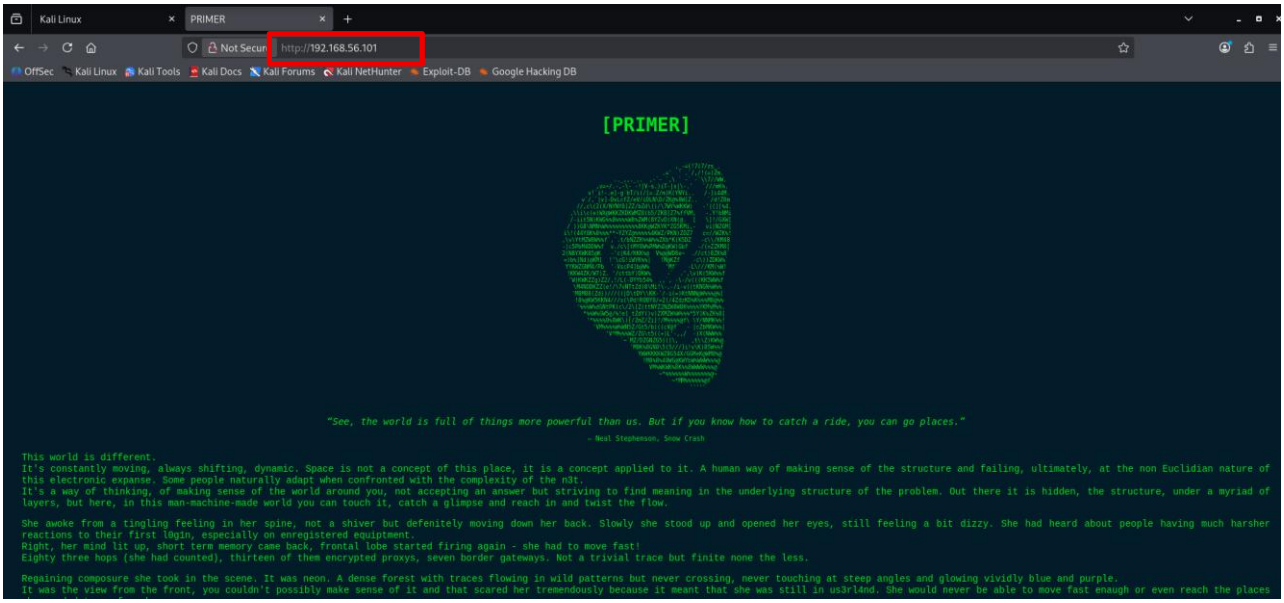


Figure 3: PRIMER landing page

Result: The landing page presents the PRIMER theme and narrative content, but no obvious account credentials.

Interpretation: The site is intentionally hiding information in its structure rather than exposing a standard login workflow.

Why this matters: Source review and web enumeration are more useful than guessing credentials from the front page.

Next step: Run web-server enumeration to identify hidden paths.

Step 4 - Enumerate the web server with Nikto

Command: nikto -h http://192.168.56.101

Description: Nikto looks for interesting files, directories, server details, and common configuration issues. The important lead is robot.txt.



Figure 4: Nikto web-server enumeration

Result: Nikto identifies Apache 2.4.10 and reports /robots.txt with a disallowed entry pointing to **/4_8f14e45fceea167a5a36dedd4bea2543/**.

Interpretation: robots.txt is being used as a breadcrumb into the next chapter.

Why this matters: It gives a concrete next URL instead of forcing us to guess.

Next step: Open robots.txt directly.

5. Hidden Web Paths and Chapter Progression

Step 5 - Read robots.txt

Command / Action: curl -s http://192.168.56.101/robots.txt

Description: The browser screenshot shows the robots file directly. Its content is short: a wildcard user-agent line and one disallowed path.

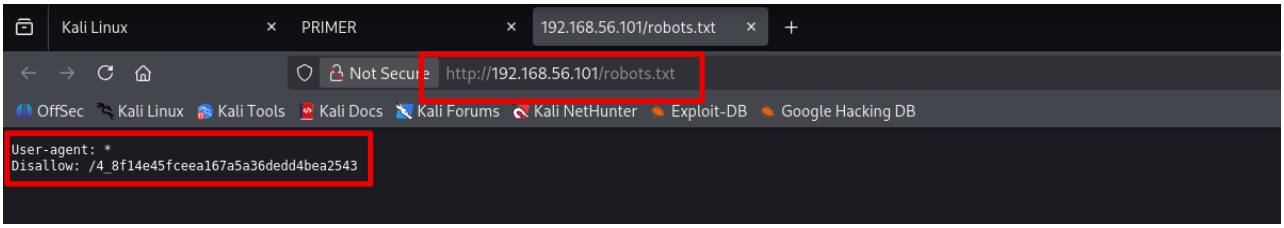


Figure 5: robots.txt disclosing the first hidden chapter path

Result: robots.txt reveals /4_8f14e45fceea167a5a36dedd4bea2543/.

Interpretation: The suffix looks like a 32-character hexadecimal value. Later evidence shows this style of suffix is an MD5 hash.

Why this matters: This is the first concrete hidden route beyond the landing page.

Next step: Visit the disclosed chapter path.

Step 6 - Open the disclosed chapter

Command / Action: Browse to http://192.168.56.101/4_8f14e45fceea167a5a36dedd4bea2543/

Description: The chapter page is titled [:( ) { :|:&;};] and ends with an EOF marker. The page is part of the story-driven path rather than a normal web directory.



Figure 6: Chapter page reached through robots.txt

Result: The hidden chapter loads successfully and continues the story.

Interpretation: The robots.txt path was intentional and is part of the challenge progression.

Why this matters: It confirms that hidden paths are being used as chapter gates.

Next step: Follow the next chapter path shown in the supplied screenshot sequence.

Step 7 - Enter the __init__ chapter

Command / Action: Browse to the next numeric/MD5-style chapter path

Description: The [__init__] page provides the narrative setup and includes a reference to identifiers and security layers, encouraging the reader to notice structural patterns.

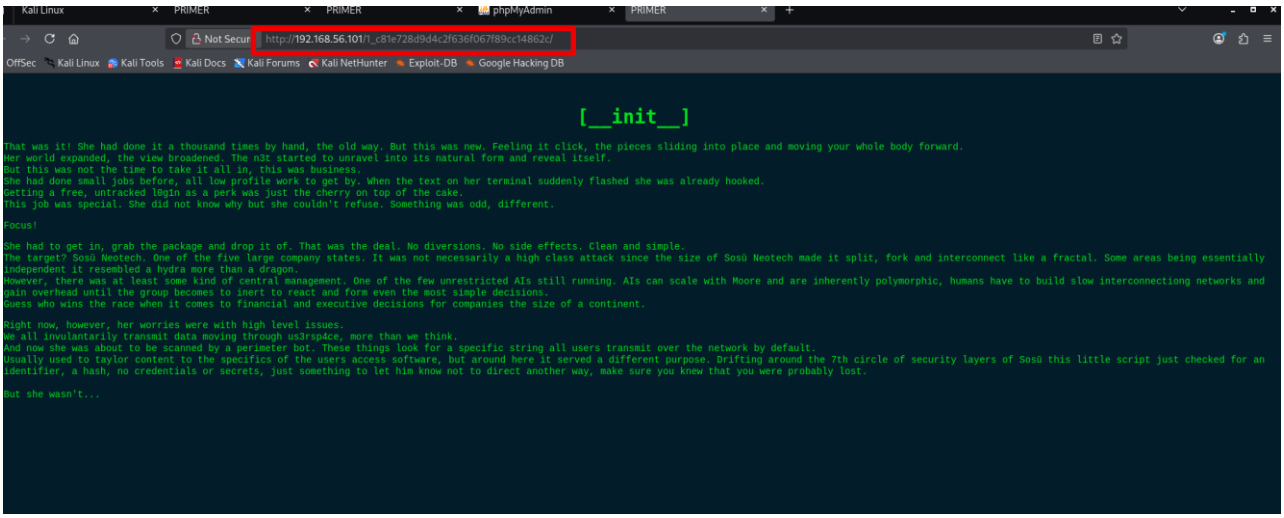


Figure 7: __init__ chapter and embedded challenge clues

Result: The page loads and supplies narrative context plus a hash-like clue.

Interpretation: Visible numbers and hashes are meant to be treated as navigation information, not just story prose.

Why this matters: The challenge rewards careful reading of page source and URL patterns.

Next step: Continue to the next chapter and watch the node numbering carefully.

Step 8 - Enter the FORK chapter and identify Node 6

Command / Action: Browse to 5_6512bd43d9caa6e02c990b0a82652dca/

Description: The FORK page explicitly displays 6_c51ce410c124a10e0db5e4b97fc2af39 as the next node.

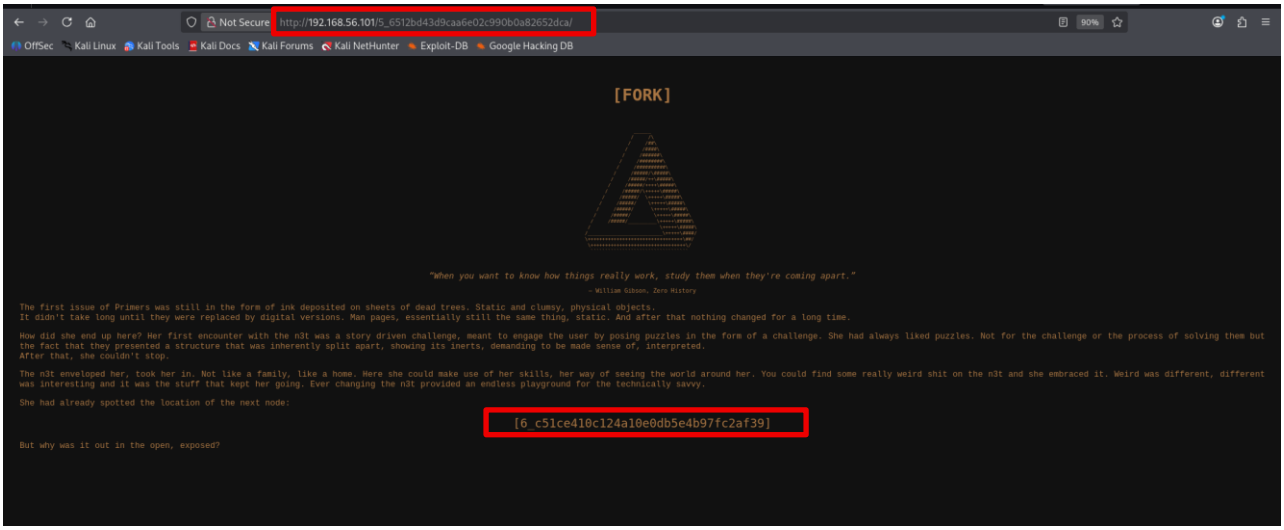


Figure 8: FORK chapter revealing the Node 6 path

Result: The next required page is **/6_c51ce410c124a10e0db5e4b97fc2af39/.**

Interpretation: The challenge has moved from passive storytelling to an explicit input gate.

Why this matters: Node 6 introduces a client-side JavaScript password check.

Next step: Open Node 6 and inspect its prompt/source.

6. Client-Side Password Gates

Step 9 - Bypass the Node 6 JavaScript password check

Command: Open /6_c51ce410c124a10e0db5e4b97fc2af39/ and inspect the page source

Description: The prompt asks for input. The JavaScript compares a substring of the entered value with the literal string Ikdf076. Because the check uses substr(2,7), Ikdf076 must begin at the third character.

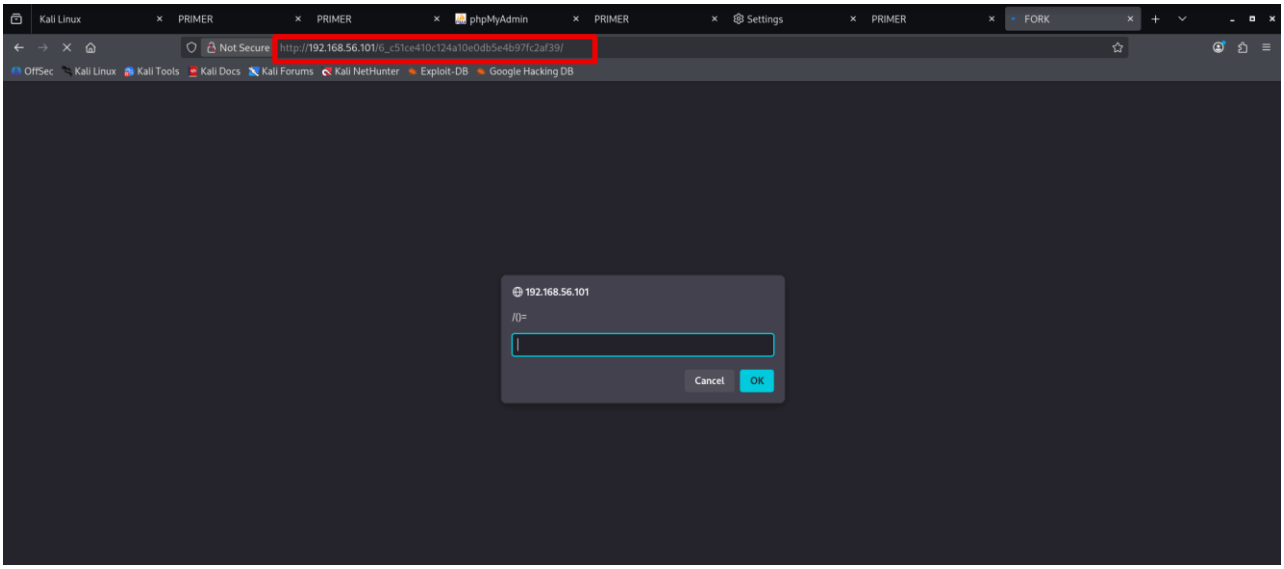


Figure 9: Node 6 input prompt

Password entered: `--Ikdf076`

Interpretation: This is not server-side authentication. The browser performs a predictable check that is visible in the JavaScript.

Why this matters: Client-side password logic should never be treated as a secure secret.

Next step: Continue to the page returned by Node 6.

Step 10 - Observe the Node 6 result page

Command / Action: Continue after the Node 6 prompt

Description: The resulting page is labeled [0xC00007B] and continues the story.

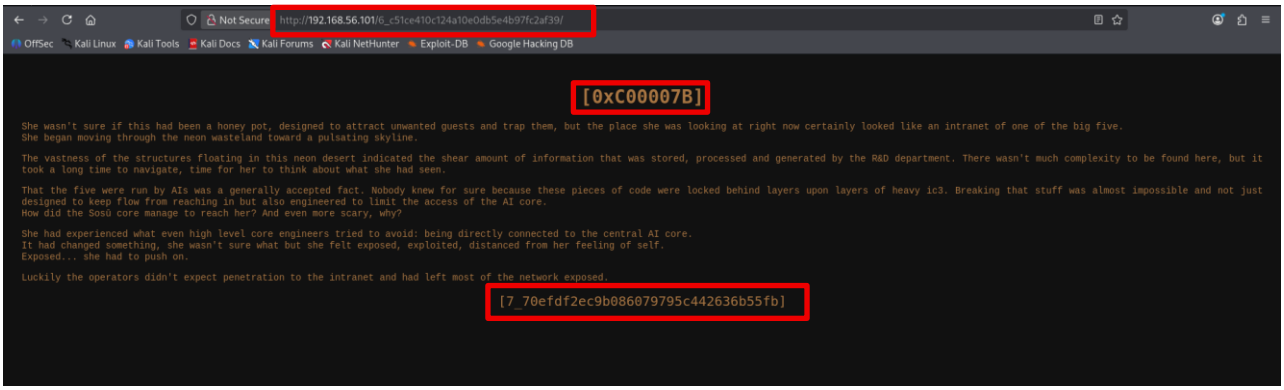


Figure 10: Successful Node 6 transition

Result: The challenge advances beyond Node 6.

Interpretation: The value satisfied the real page logic rather than merely looking plausible.

Why this matters: Successful navigation proves the client-side gate was solved correctly.

Next step: Move to Node 7 and inspect its JavaScript.

Step 11 - Inspect Node 7 source for the second password

Command / Action: View the Node 7 page source / developer tools

Description: The developer-tool screenshots show the password-checking JavaScript and two MD5 values. The important target hash is **0d28cba0bd4f26e16d766000d27e49fa**.

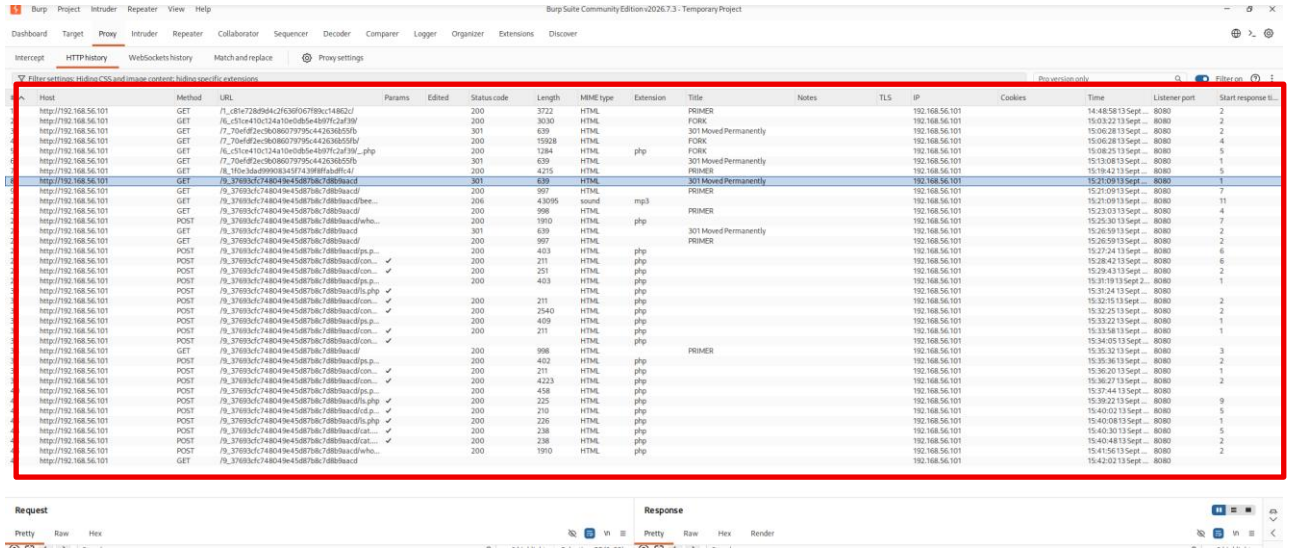


Figure 11: Browser developer tools and Node 7 JavaScript

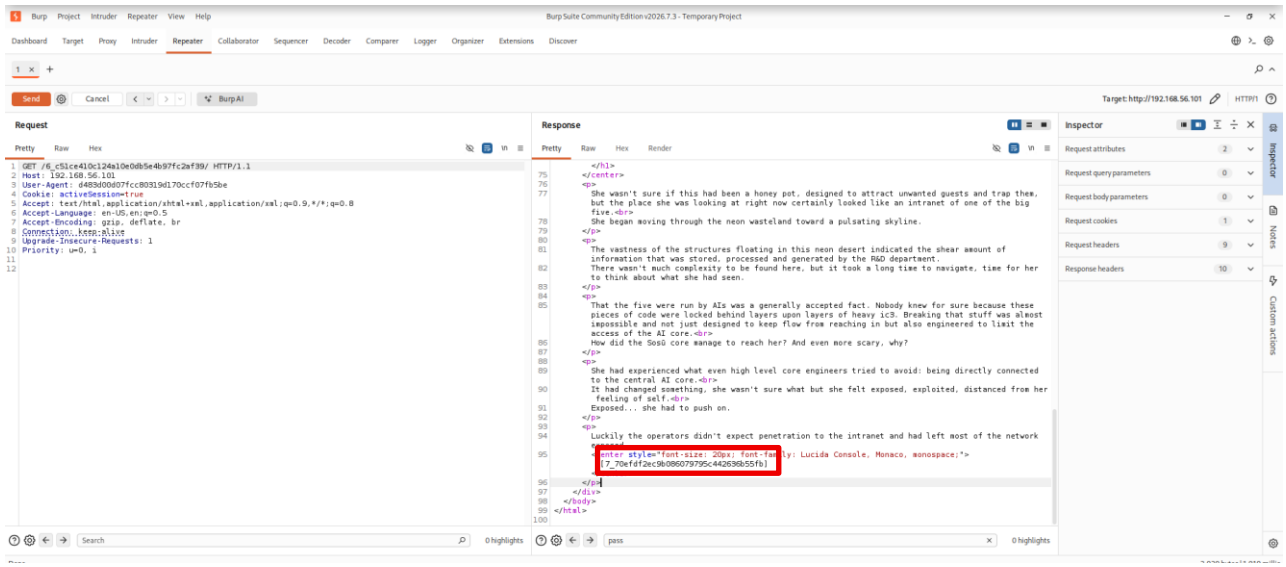


Figure 12: Source code showing the password-check logic

Result: The page hashes the user's input with MD5 and compares it against the embedded target hash.

Interpretation: The secret is embedded in the browser code and can therefore be recovered by reversing the hash.

Why this matters: This illustrates why secrets and password checks should live on the server, not in client-side code.

Next step: Enter the recovered plaintext at the Node 7 prompt.

Step 12 - Enter the Node 7 password

Command / Action: Enter the recovered password at the Node 7 prompt

Description: The prompt appears on the 7_70efdf2ec9b086079795c442636b55fb node.

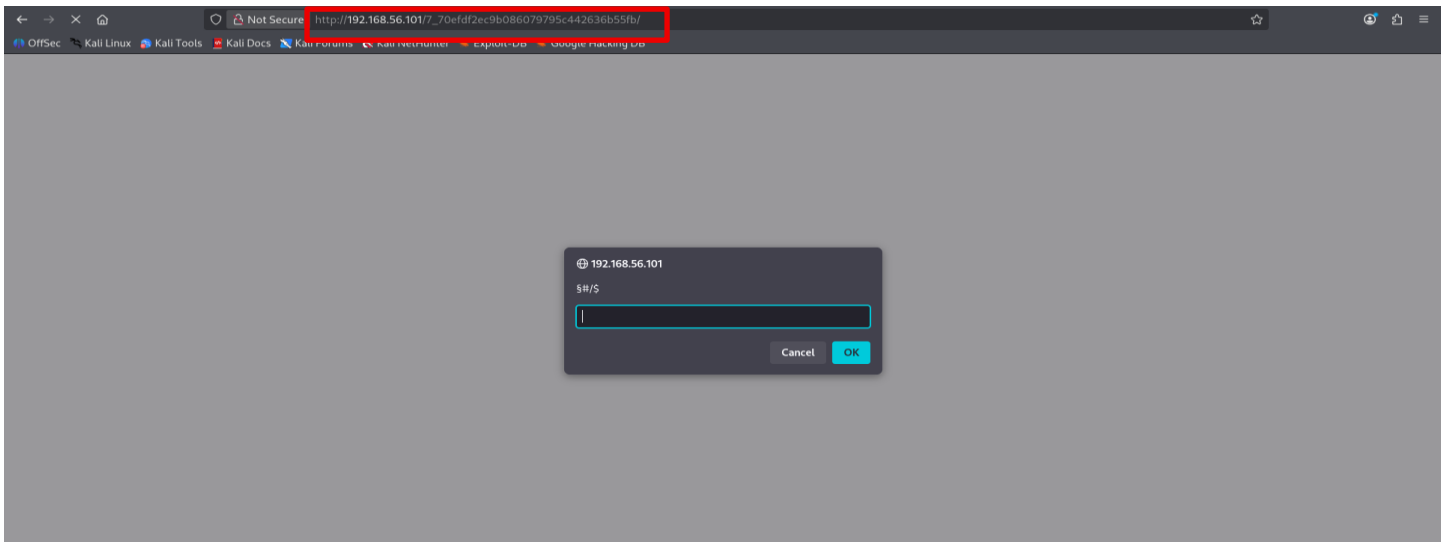


Figure 13: Node 7 password prompt

Password entered: GOD

Interpretation: The second browser-side password gate is now cleared.

Why this matters: The success proves the value matched the actual page logic.

Next step: Continue to the next chapter.

Step 13 - Confirm the Node 7 success page

Command / Action: Continue after the Node 7 prompt

Description: The page changes to the next chapter theme and continues the story.

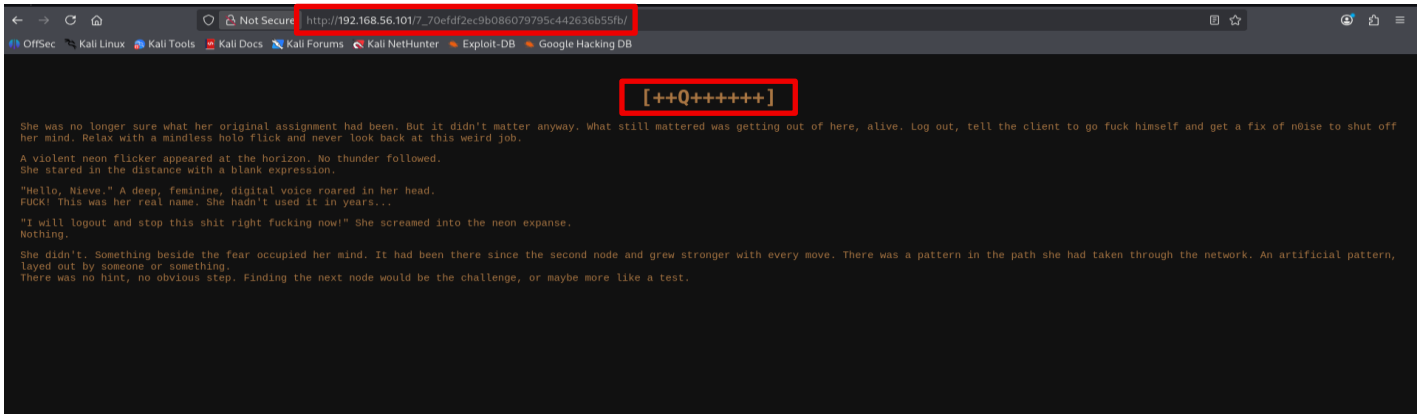


Figure 14: Successful transition after Node 7

Result: The challenge proceeds into the next puzzle stage.

Interpretation: The challenge has moved from password-window analysis into pattern recognition.

Why this matters: The next clue is the relationship between node numbers and MD5 suffixes.

Next step: Compute the relevant hashes and identify the numeric sequence.

7. Recognizing the Prime-Number URL Pattern

Step 14 - Recover the hidden numeric pattern from the MD5 suffixes

Command : echo -n 17 | md5sum
echo -n 19 | md5sum

Description: The terminal screenshots show MD5 calculations for prime-number candidates. Earlier URL suffixes decode to prime values, so the node number and MD5 value form a deliberate pair.

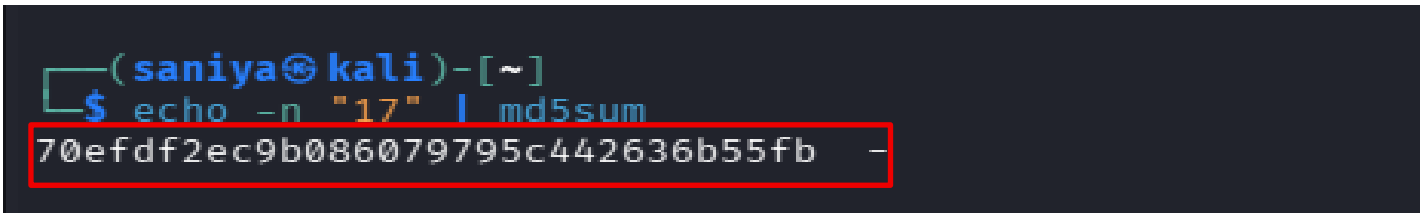


Figure 15: MD5 calculation for a prime-number node



Figure 16: MD5 calculation for the next prime-number node

Result: The screenshots support the sequence of prime plaintext values being converted to MD5 for the node URLs.

Interpretation: The URL format is effectively number_MD5(number), so the next node can be derived from the next prime rather than guessed.

Why this matters: Recognizing the pattern turns random-looking URLs into a deterministic navigation scheme.

Next step: Use the next prime-derived node to reach the simulated terminal.

Step 15 - Open the next node and reach the simulated terminal

Command / Action: Browse to the next prime/MD5-style node; then continue to the terminal page

Description: The [KS(x)<=l(x)+4] page is followed by a simulated terminal. The terminal supports a small set of custom commands including ls, ps, cat, connect, hash, and decode.

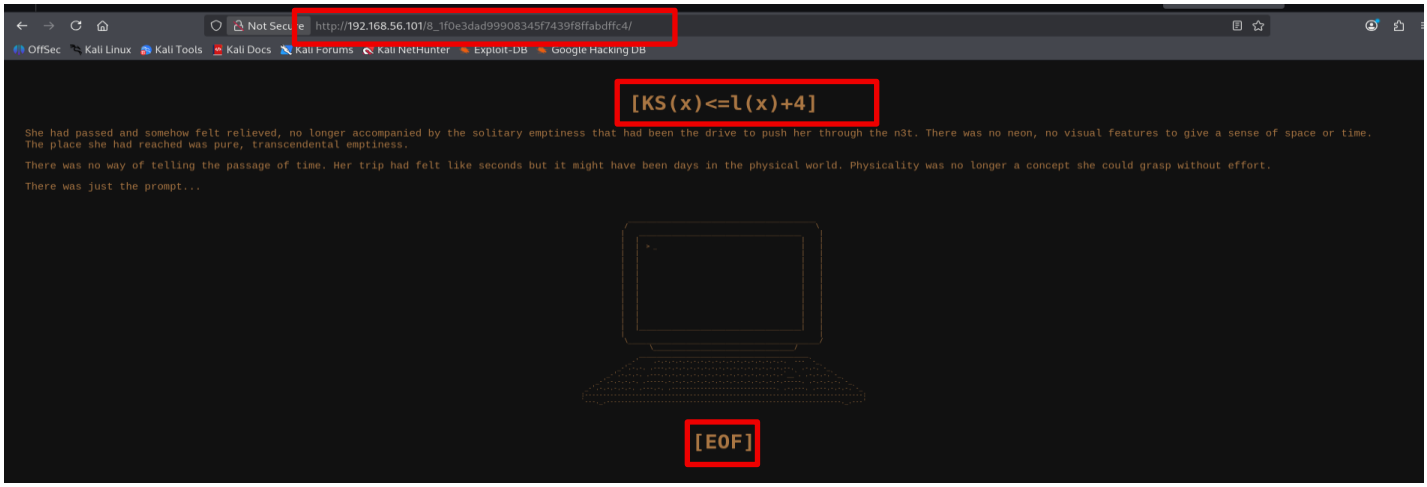


Figure 17: KS(x)<=l(x)+4 chapter

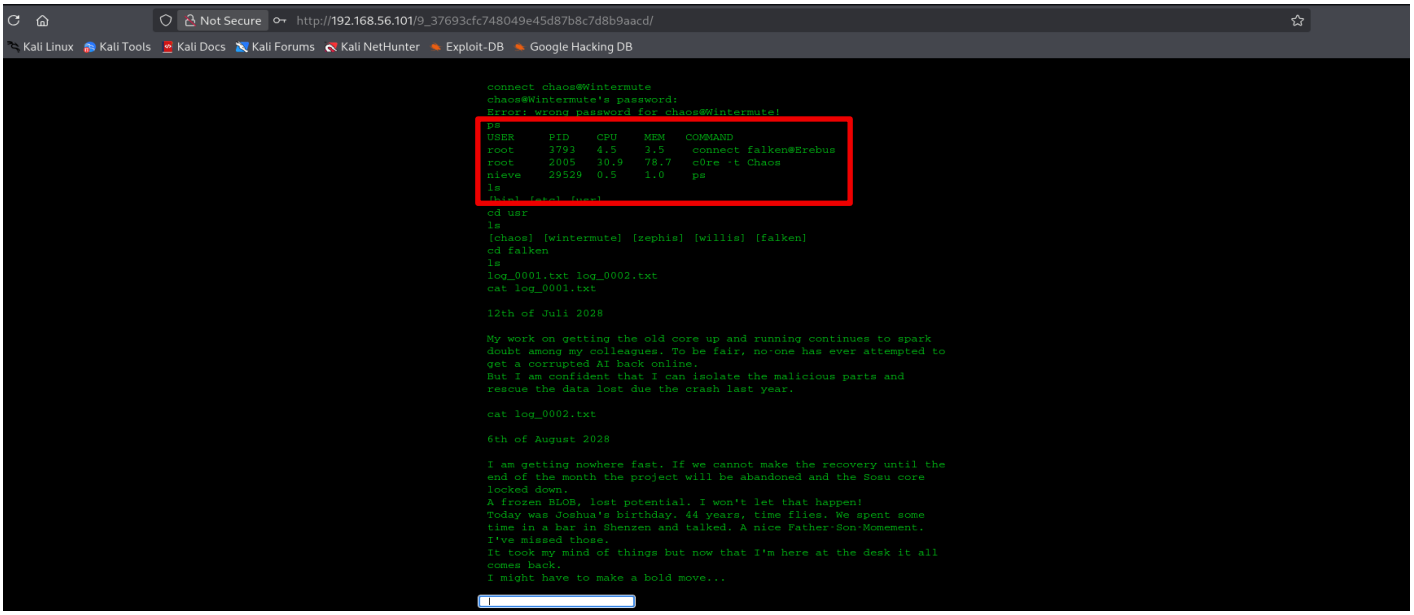


Figure 18: Simulated terminal inside the browser

Result: A browser-based terminal becomes available.

Interpretation: The terminal is part of the challenge application, not a normal Linux shell. Its commands are puzzle mechanics implemented by the site.

Why this matters: From here, enumeration means reading the simulated filesystem and following the named connections.

Next step: Use ps and ls to discover the next node and the relevant users.

8. Pivot to Erebus

Step 16 - Use process information to discover the next connection

Command / Action: ps

Description: The ps output shows a connection involving falken@Erebus while the current user is nieve.

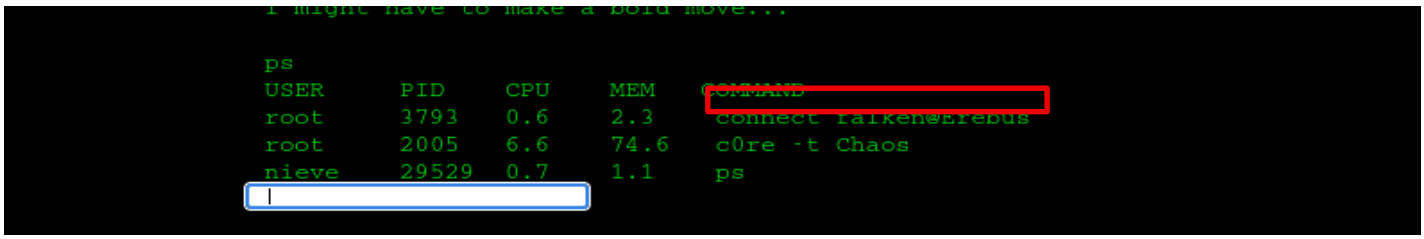


Figure 19: ps output reveals the falken@Erebus connection

Result: The next reachable system is Erebus, and the terminal asks for a password for falken.

Interpretation: The process list functions as a roadmap to the next pivot.

Why this matters: Running process information can reveal users, services, command lines, and connected systems.

Next step: Authenticate to falken@Erebus.

Step 17 - Authenticate to falken@Erebus

Command / Action: connect falken@Erebus

Description: The screenshot shows the actual password prompt. The password is derived from a log clue in which Joshua is 44 years old on 6 August 2028, giving a 1984 birth year.

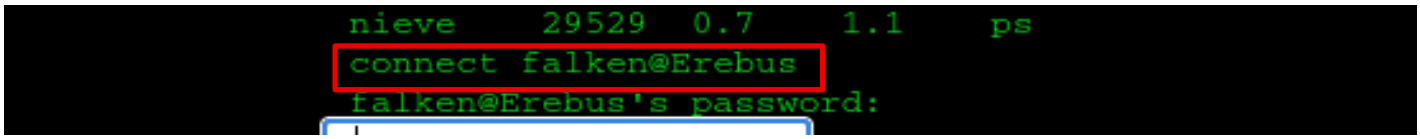


Figure 20: Password prompt for falken@Erebus

Password entered: `joshua1984`

Result: The connection is established after joshua1984 is entered.

Interpretation: The credential is derived from internal information rather than blindly guessed.

Why this matters: Readable internal notes can reveal reusable credentials.

Next step: Enumerate Erebus and inspect falken's files.

Step 18 - Enumerate Erebus and inspect falken's files

Command / Action: ls; cd usr; ls; cd falken; ls; cat log_0001.; cat log_0002.

Description: The Erebus environment contains /usr entries and a falken directory with log files. The logs contain dates and personal information, including Joshua's birthday clue.



Figure 21: Erebus connection established

Result: The falken logs explain the clue that produced joshua1984 and provide context for the next stage.

Interpretation: The challenge hides the useful information in user-owned story logs rather than an obvious password file.

Why this matters: Enumeration followed by file reading is the intended discovery workflow.

Next step: Inspect the encoded data and decode it until the next password clue becomes readable.

Step 19 - Review and decode the falken log data

Command: cat log_0001.; cat log_0002.; decode ...

Description: The supplied screenshots show long encoded blocks and multiple decode attempts. The important progression is that the data changes encoding layers before revealing readable text.

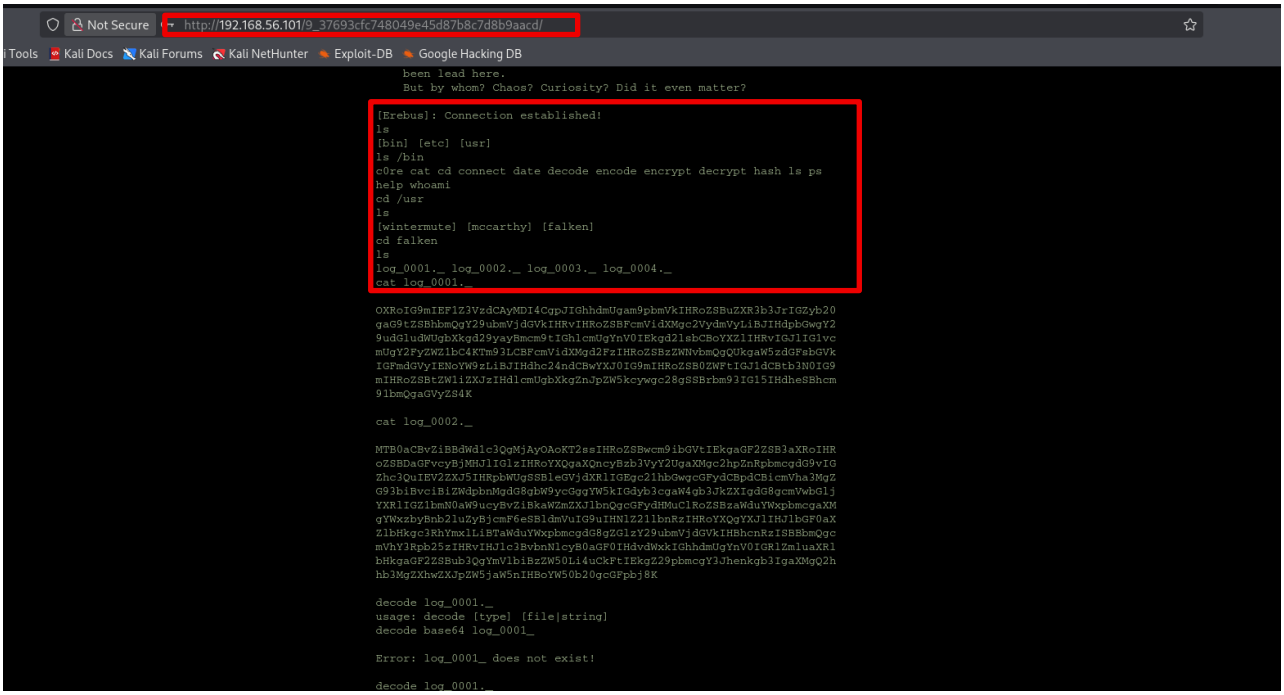


Figure 22: Encoded falken data and file listing

```
ls
log_0001._ log_0002._ log_0003._ log_0004._
cat log_0001._

OXRoIG9mIEF1Z3VzdCAyMDI4CgpJIGhhdmUgam9pbmVkIHRoZSBuZXR3b3JrIGZyb20
gaG9tZSBhbmQgY29ubmVjdGVkIHRvIHRoZSBFcmVidXMgc2VydmVyLiBJIHdpbGwgY2
9udGludWUgbXkgd29yayBmcm9tIGhlcmUgYnV0IEkgd2lsbCBoYXZlIHRvIGJlIG1vc
mUgY2FyZWZ1bC4KTm93LCBFcmVidXMgd2FzIHRoZSBzZWNvbmQgQUkgaW5zdGFsbGVk
IGFmdGVyIENoYW9zLiBJIHdhc24ndCBwYXJ0IG9mIHRoZSB0ZWFtIGJ1dCBtb3N0IG9
mIHRoZSBtZW1iZXJzIHdlcmUgbXkgZnJpZW5kcywgc28gSSBrbm93IG15IHdheSBhcm
91bmQgaGVyZS4K

cat log_0002._

MTB0aCBvZiBBdWd1c3QgMjAyOAoKT2ssIHRoZSBwcm9ibGVtIEkgaGF2ZSB3aXRoIHR
oZSBDaGFvcyBjMHJlIGlzIHRoYXQgaXQncyBzb3VyY2UgaXMgc2hpZnRpbmcgdG9vIG
Zhc3QuIEV2ZXJ5IHRpbWUgSSBleGVjdXRlIGEgc21hbGwgcGFydCBpdCBicmVha3MgZ
G93biBvciBiZWdpbnMgdG8gbW9ycGggYW5kIGdyb3cgaW4gb3JkZXIgdG8gcmVwbGlj
YXRlIGZlbmN0aW9ucyBvZiBkaWZmZXJlbnQgcGFydHMuClRoZSBzaWduYWxpbmcgaXM
gYWxzbyBnb2luZyBjcmF6eSBldmVuIG9uIHN1Z2llbnRzIHRoYXQgYXJlIHJlbGF0aX
ZlbHkgc3RhYmxlLiBTaWduYWxpbmcgdG8gZGlzY29ubmVjdGVkIHBhcnRzISBBbmQgc
mVhY3Rpb25zIHRvIHJlc3BvbnNlcyB0aGF0IHdvdWxkIGhhdmUgYnV0IGRlZmluaXRl
bHkgaGF2ZSBub3QgYmVlbiBzZW50Li4uCkFtIEkgZ29pbmcgY3Jhenkgb3IgaXMgMgQ2h
hb3MgZXhwZXJpZW5jaW5nIHBoYW50b20gcGFpbj8K

decode log_0001._
usage: decode [type] [file|string]
decode base64 log_0001._

Error: log_0001._ does not exist!

decode log_0001._
usage: decode [type] [file|string]
decode base64 log_0001._

9th of August 2028

I have joined the network from home and connected to the Erebus
server. I will continue my work from here but I will have to be
more careful.
Now, Erebus was the second AI installed after Chaos. I wasn't part
of the team but most of the members were my friends, so I know my
way around here.
```

Figure 23: Encoded logs and decoding attempts

```
way around here.

decode log_0002._
usage: decode [type] [file|string]
decode log_0003._
usage: decode [type] [file|string]
decode gz log_0003._

12th of August 2028

I think they might be on to me and I can only change the encoding
so often.
I will have to do something reckless... but not from here, they are
already too close.

decode gz log_0004._

PCH ybnq vapernfrq abgvprnoyl, fbzrguvat vf tbvat gb unccra naq V
jvyy abg or nebhaq gb jvgarff... GevivnyM3e0 frrzf gb or dhvrg,
qba'g xabj jung'f tbvat ba bire gurer ohg vg pna'g or jbefr guna
orvat genprq qbja ol znvaynaq fcbbxf. Svefg V arrq fbzr perqf,
gubhtu. Uzz, GevivnyM3e0... gung erzvaqf zr bs fbzrbar, yrg'f whfg
ubcr gubfr thlf nera'g zngu trrxf.
```

Figure 24: Additional decoding step revealing a second-stage clue

```
ubcr gubfr thlf nera'g zngu trrxf.

decode rot13 "PCH ybnq vapernfrq abgvprnoyl, fbzrguvat vf tbvat gb
unccra naq V jvyy abg or nebhaq gb jvgarff... GevivnyM3e0 frrzf gb
or dhvrg, qba'g xabj jung'f tbvat ba bire gurer ohg vg pna'g or
jbefr guna orvat genprq qbja ol znvaynaq fcbbxf. Svefg V arrq fbzr
perqf, gubhtu. Uzz, GevivnyM3e0... gung erzvaqf zr bs fbzrbar,
yrg'f whfg ubcr gubfr thlf nera'g zngu trrxf."

CPU load increased noticeably, something is going to happen and I
will not be around to witness... TrivialZ3r0 seems to be quiet,
don't know what's going on over there but it can't be worse than
being traced down by mainland spooks. First I need some creds,
though. Hmm, TrivialZ3r0... that reminds me of someone, let's just
hope those guys aren't math geeks.
```

Figure 25: ROT13 text and the hint about being math geeks

Result: The decoded text contains the mathematical clue Riemann and points toward TrivialZ3r0.

Interpretation: The challenge teaches a layered decoding workflow: identify the encoding, transform the data, and then read the resulting clue.

Why this matters: The next credential is a thematic keyword rather than a random secret.

Next step: Use Riemann as the password for falken@TrivialZ3r0.

9. Pivot to TrivialZ3r0

Step 20 - Authenticate to falken@TrivialZ3r0

Command: connect falken@TrivialZ3r0

Description: The terminal requests the falken password for TrivialZ3r0. The decoded logs point to the mathematical name Riemann.

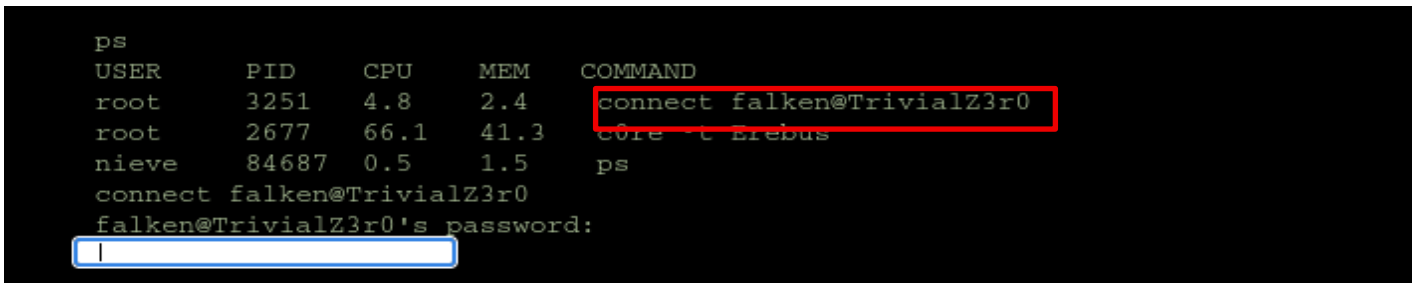


Figure 26: Password prompt for falken@TrivialZ3r0

Password entered: `Riemann`

Result: The connection proceeds into TrivialZ3r0.

Interpretation: The hostname itself is a clue to the mathematical concept used as the password.

Why this matters: A known clue can be reused exactly to authenticate to the next node.

Next step: Enumerate the passwd directory and identify the hash belonging to chaos.

Step 21 - Enumerate the password store on TrivialZ3r0

Command: ls; cd passwd; ls; cat falken; cat chaos; cat mccarthy

Description: The screenshot shows a simplified passwd directory containing falken, chaos, and mccarthy. Each entry is represented by an MD5-looking value.

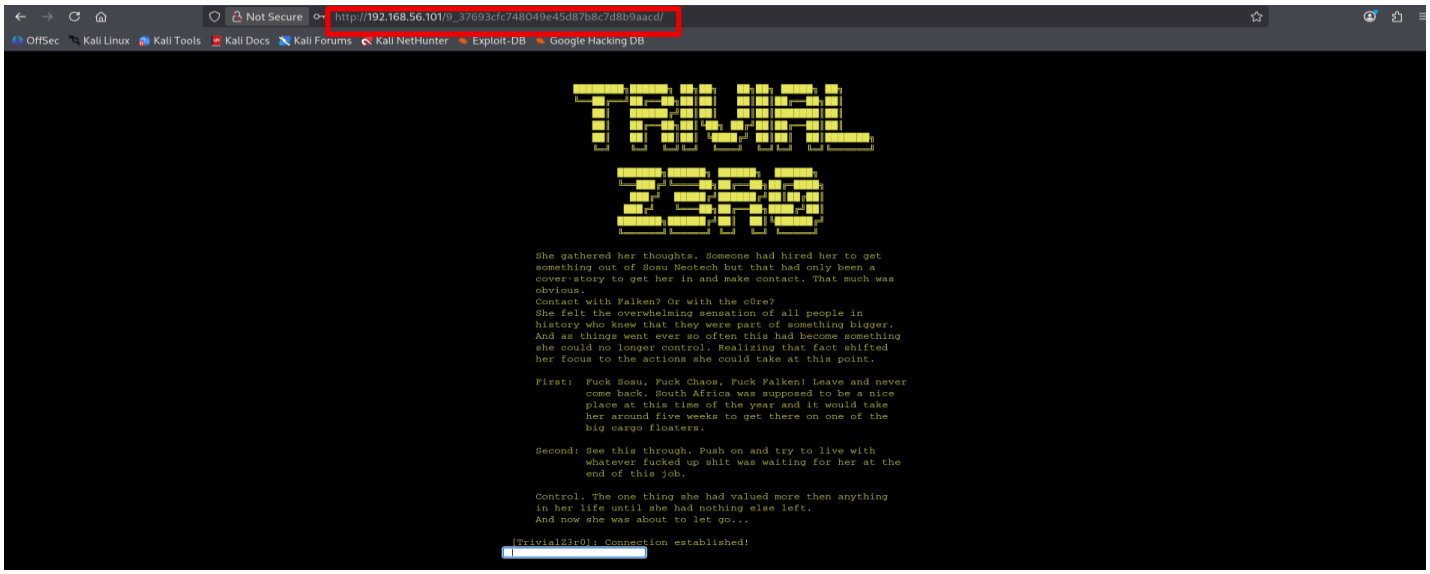


Figure 27: TrivialZ3r0 environment and passwd directory

Result: The chaos entry contains **85241de03d1254ac40274b02caafcd99.**

Interpretation: The ps output tells us chaos is connected to Wintermute, so the chaos hash is the relevant credential to recover.

Why this matters: This narrows the cracking target to the account that actually leads forward.

Next step: Recover the plaintext corresponding to the chaos MD5 hash.

Step 22 - Recover the chaos password

Command: Crack MD5: 85241de03d1254ac40274b02caafcd99

Description: The verified challenge walkthrough identifies the chaos hash as the decimal representation of Euler's number.



Figure 28: MD5 password hashes for falken, chaos, and mccarthy

Result: The chaos password is 2.718281828459045.

Interpretation: Only the chaos value is needed because the process information already identifies chaos as the connection to Wintermute.

Why this matters: The challenge combines hash cracking with process enumeration so that the right hash is selected first.

Next step: Use the recovered chaos password to connect to Wintermute.

10. Pivot to Wintermute and Recover the Final Credentials

Step 23 - Connect to Wintermute

Command: connect chaos@Wintermute

Description: The terminal requests the chaos password. This is the final authentication gate before the last credential file becomes available.

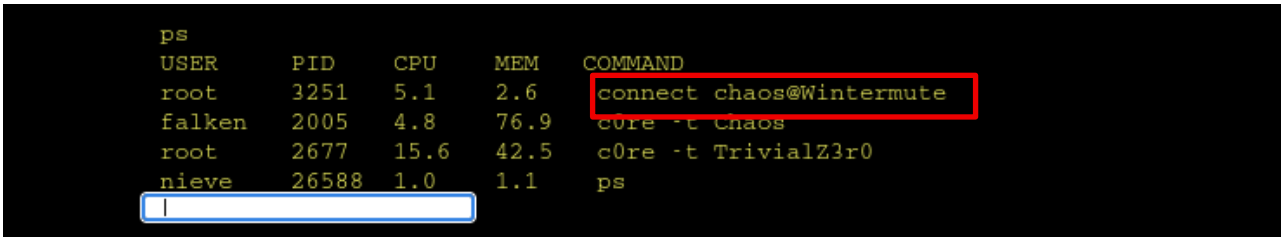


Figure 29: Password prompt for the chaos account / Wintermute pivot

Password entered: `2.718281828459045`

Result: The connection advances to Wintermute.

Interpretation: The same password recovered from the hash is reused exactly as the challenge expects.

Why this matters: This is the third pivot in the simulated network chain: base terminal -> Erebus -> TrivialZ3r0 -> Wintermute.

Next step: Enumerate Wintermute for the final file.

Step 24 - Enumerate the final Wintermute environment

Command: ps; ls; cat nieve

Description: Wintermute is intentionally sparse. The screenshot shows the chapter page followed by an established connection and a single user file named nieve.

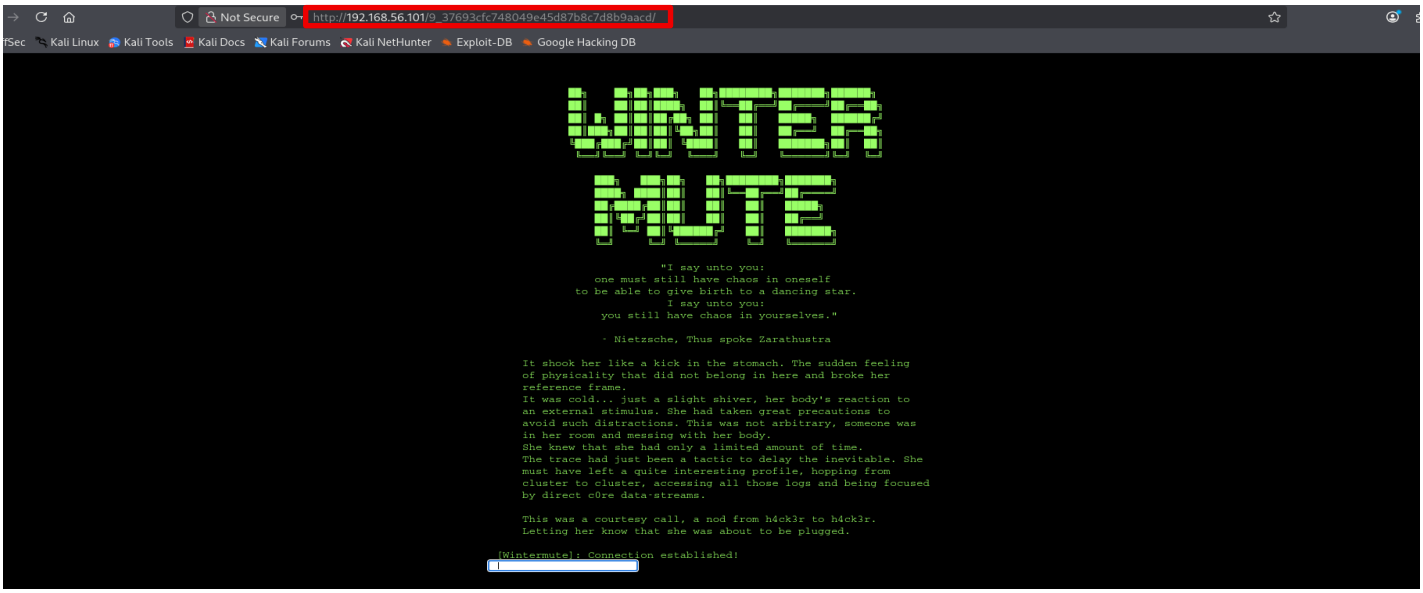


Figure 30: Wintermute chapter page

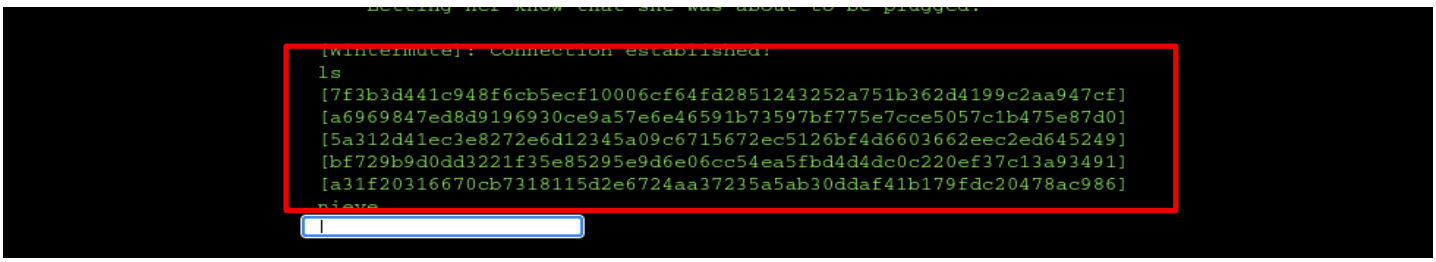


Figure 31: Wintermute connection and final file contents

Result: The final user file is nieve, and reading it reveals explicit credentials for the next host.

Interpretation: The challenge has stopped hiding the answer behind another cryptographic puzzle; the final account information is written directly in the file.

Why this matters: Good enumeration ultimately reduces a complex chain to one useful artifact.

Next step: Record the final credentials and complete the challenge.

Step 25 - Read the final credentials from nieve

Command: cat nieve

Description: The final screenshot contains the closing story and, at the bottom, a credential block.

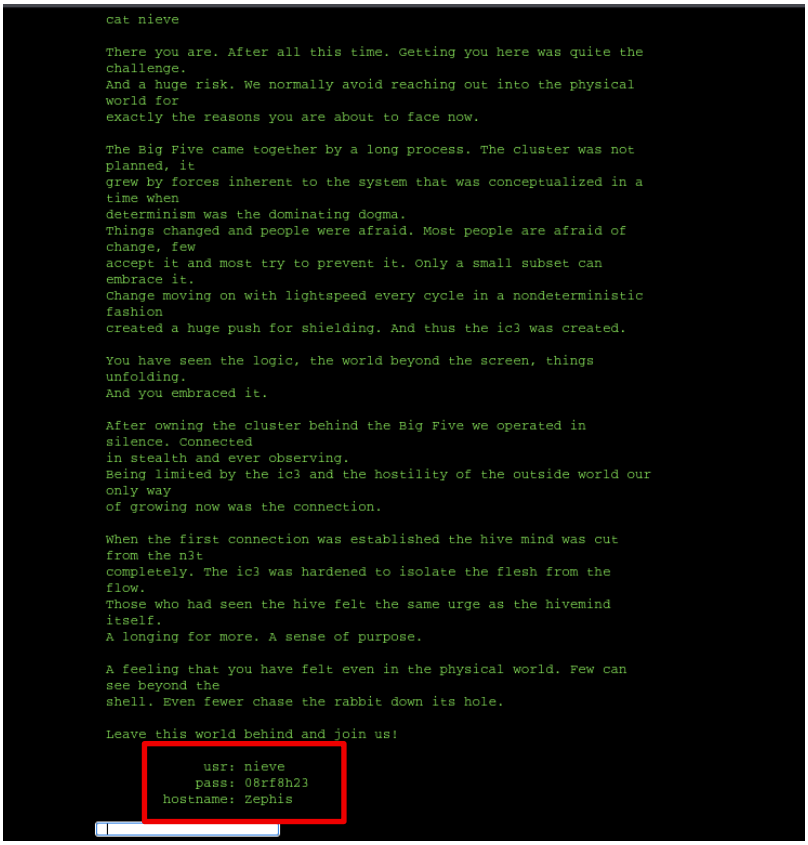


Figure 32: Final file revealing the nieve credentials

Password entered: `08rf8h23`

The final file explicitly shows user nieve, password 08rf8h23, and hostname Zephis.

Result: The file shows user nieve, password 08rf8h23, hostname Zephis.

Interpretation: The final objective is reached because the challenge openly discloses the exact credentials needed to connect onward.

Why this matters: This completes the puzzle chain from hidden URLs to the final named host.

Next step: Use nieve / 08rf8h23 on Zephis.

Step 26 - Complete the PRIMER sequence

Command: connect nieve@Zephis

Description: The closing screenshot returns to the PRIMER theme and confirms that the final stage has been reached.

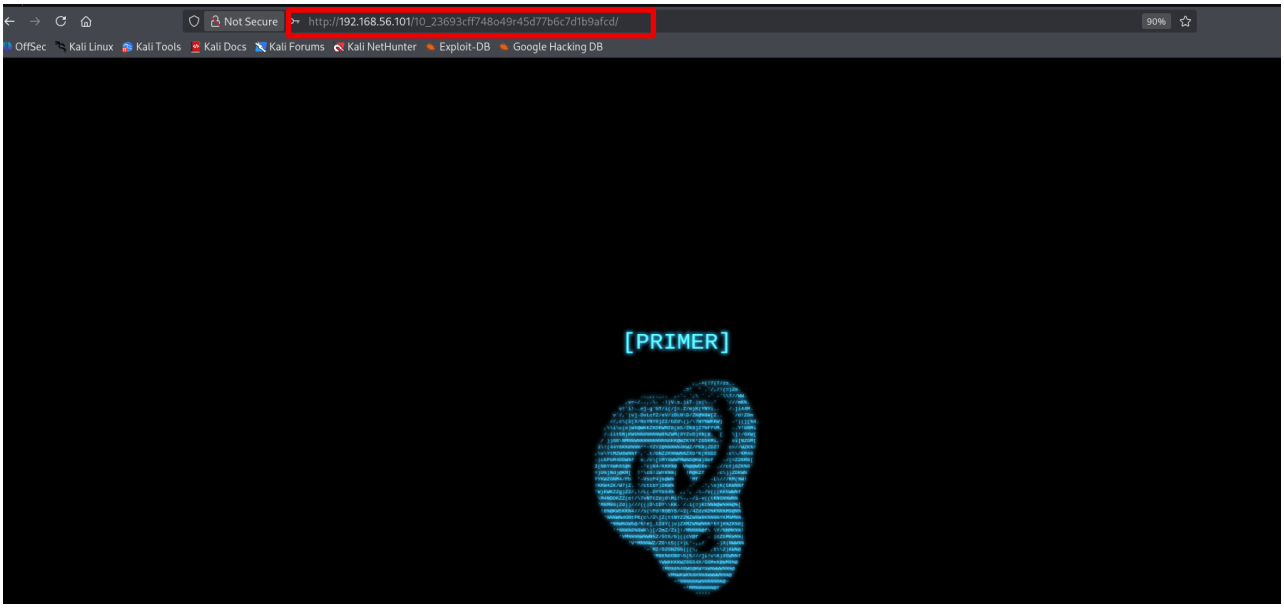


Figure 33: PRIMER closing screen

Result: The PRIMER sequence completes successfully.

Interpretation: The screenshot chain is internally consistent from target discovery to final credentials.

Why this matters: The report now records the complete password and pivot sequence for repeatability.

Next step: Use the summary table as the quick reference.

11. Credential Summary

The table below collects every password associated with a required authentication prompt in the supplied screenshot sequence. The detailed sections above explain where each value comes from.

Node / Account	Value
Node 6 JavaScript check	--Ikdf076
Node 7 JavaScript check	GOD
falken@Erebus	joshua1984
falken@TrivialZ3r0	Riemann
chaos@Wintermute	2.718281828459045
nieve@Zephis	08rf8h23

Why these passwords work

Node 6 is satisfied by a substring test in the browser JavaScript, so the important pattern is Ikdf076 beginning at the third character. Node 7 uses an MD5 comparison in JavaScript, and GOD is the plaintext represented by the expected hash. The later passwords are puzzle-derived from visible notes and hashes: joshua1984 comes from the Joshua/1984 birthday clue, Riemann comes from the decoded mathematical hint, 2.718281828459045 comes from the chaos MD5 hash, and 08rf8h23 is explicitly printed in the final nieve file.

12. Conclusion

PRIMER 1.0.1 is deliberately built as a sequence of small discoveries rather than a single exploit. The supplied evidence demonstrates the whole path: identify the target, enumerate the web server, follow hidden paths, read client-side JavaScript, recognize MD5 and prime-number patterns, use the simulated terminal to enumerate users and processes, derive pivot credentials from logs and hashes, and finally read the last credential file.

The strongest practical lesson is that each stage becomes easier once the previous stage is understood. The report therefore keeps every screenshot in its original state and places the explanation around it, without drawing boxes or changing the evidence. Passwords are shown below each actual password-input screenshot so the document can be followed step by step without breaking the screenshot chain.

ummary presentation.